Intermediate — a file-upload app — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: half of real AI applications take a file as input, not typed text — a PDF invoice to extract fields from, a resume to match against a job. This is the file-upload half of the UI that those apps actually need. **If not:** your app would only ever work on text someone pastes in by hand, and document-driven projects like InvoiceIQ or JobMatch would have no real front door for users to reach them through.

The pipeline

```
# practice/pipeline.py — stands in for a real extractor/analyzer
def analyze(text: str) -> dict:
    return {"chars": len(text), "has_total": "total" in text.lower()}
```

Streamlit: `st.file_uploader`

```
# practice/app_upload_streamlit.py   —   streamlit run app_upload_streamlit.py
import streamlit as st

from pipeline import analyze

st.title("Document Analyzer")

uploaded = st.file_uploader("Upload a .txt or .pdf", type=["txt", "pdf"])
if uploaded is not None:
    if uploaded.name.lower().endswith(".pdf"):
        # save to a temp path, then use Doc08's PyPDFLoader on it
        import tempfile
        from langchain_community.document_loaders import PyPDFLoader
        with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as tmp:
            tmp.write(uploaded.read())
            path = tmp.name
        pages = PyPDFLoader(path).load()
        texts = []
        for page in pages:
            texts.append(page.page_content)
```

```
        text = "\n".join(texts)
    else:
        text = uploaded.read().decode()
    st.write(analyze(text))
```

`st.file_uploader` returns a file-like object (or `None` before anything is uploaded); `.read()` gives the bytes. For a PDF you save those bytes to a temp file and hand the path to `PyPDFLoader` (Doc08) — for text you just `.decode()`.

FastAPI: `UploadFile`

```
# practice/app_upload_fastapi.py   —   uvicorn app_upload_fastapi:app --reload
from fastapi import FastAPI, UploadFile

from pipeline import analyze

app = FastAPI()

@app.post("/upload")
async def upload(file: UploadFile):
    name = (file.filename or "").lower()
    if not name.endswith((".txt", ".md")):
        return {"error": "only .txt/.md supported here"}
    data = await file.read()
    return {"filename": file.filename, "result": analyze(data.decode())}
```

`UploadFile` gives `file.filename` and the bytes via `await file.read()`. Validate the type before doing anything — an upload is untrusted input.

Testing the FastAPI upload (no server)

```
# practice/test_upload.py   —   python -m practice.test_upload
from fastapi.testclient import TestClient

from app_upload_fastapi import app

def test_upload_and_reject() -> None:
    client = TestClient(app)
    good = client.post(
        "/upload",
        files={"file": ("note.txt", b"invoice total 40", "text/plain")},
    ).json()
```

```
    bad = client.post(
        "/upload",
        files={"file": ("x.exe", b"bytes", "application/octet-stream")},
    ).json()
    result = "OK"
    if good["result"]["has_total"] is not True:
        result = "WRONG"
    if "error" not in bad:
        result = "WRONG"
    print(f"case 1 (accepts .txt, rejects .exe) -> {result}")

if __name__ == "__main__":
    test_upload_and_reject()
```

Expected: a `.txt` upload returns the analysis (`has_total` true for "invoice total 40"); a `.exe` is rejected with an error, not processed.

The pattern: Streamlit `st.file_uploader` → `.read()` ; FastAPI `UploadFile` → `await .read()` ; Flask `request.files["file"]` . Every one hands you bytes you save or decode, then feed to the same pipeline — the file is just another way in.